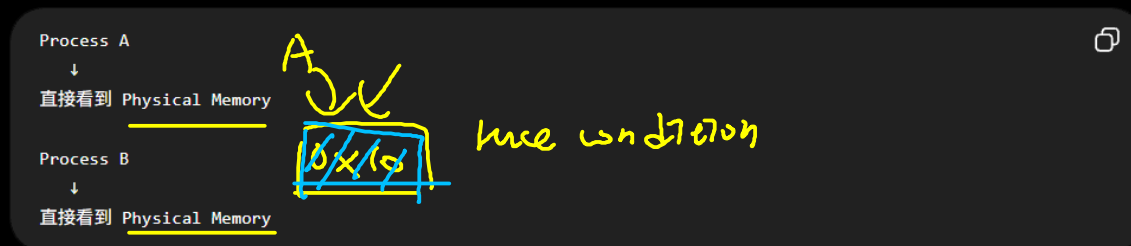


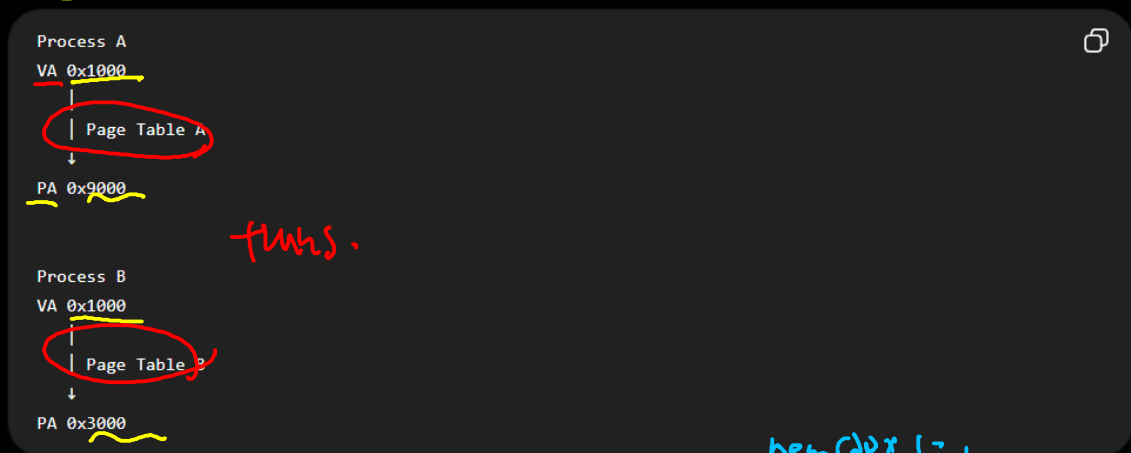
# Virtual Memory

假設沒有 VM :



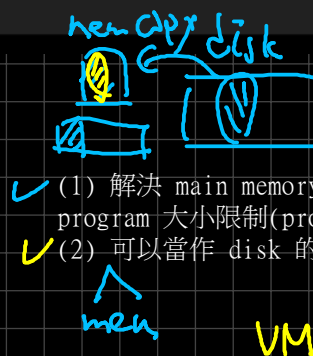
那 A、B 都得知道自己實際被 OS 放在 RAM 哪裡，而且還可能互相碰到對方的記憶體。

Virtual Memory 加進來後：



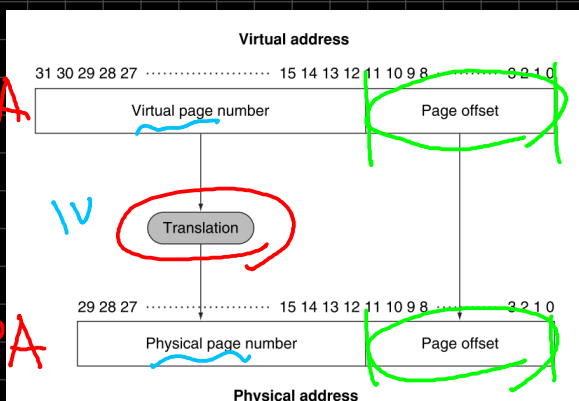
因此 VM 提供了：

- Address space isolation
- ✓ Protection
- Relocation ?
- Sharing
- ★ 讓每個 process 擁有自己的 virtual address space



- ✓ (1) 解決 main memory 過小造成 program 大小限制(program 大小可以大於整 memory)
- ✓ (2) 可以當作 disk 的 cache

VM 成立的機制：translation

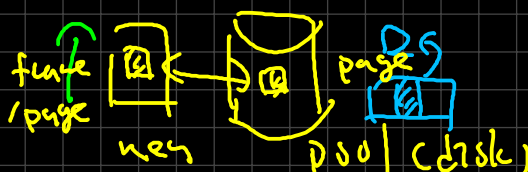


VM has basic unit = page

(8/4 slots)

cache  
basic unit

VA: Virtual Address  
PA: Physical Address  
VPN: Virtual Page Number  
PFN/PPN: Physical Frame Number/Physical Page Number



mapping

| VPN  | PPN <sup>PTE</sup> | VPN  | PPN  |
|------|--------------------|------|------|
| 0x01 | 0x27               | 0x81 | 0x8a |
| 0x13 | 0x45               | 0x9f | 0xcd |
| 0x2e | 0xe8               | 0xad | 0x7e |

VPN: Virtual Page Number

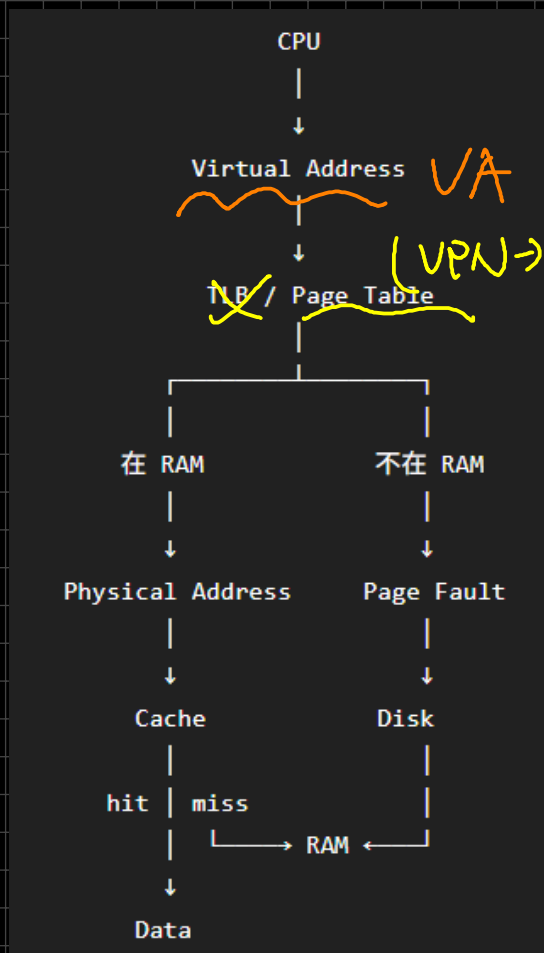
PPN/PPN: Physical Frame Number/Physical Page Number

PTE: Page Table Entry

PT: Page Table

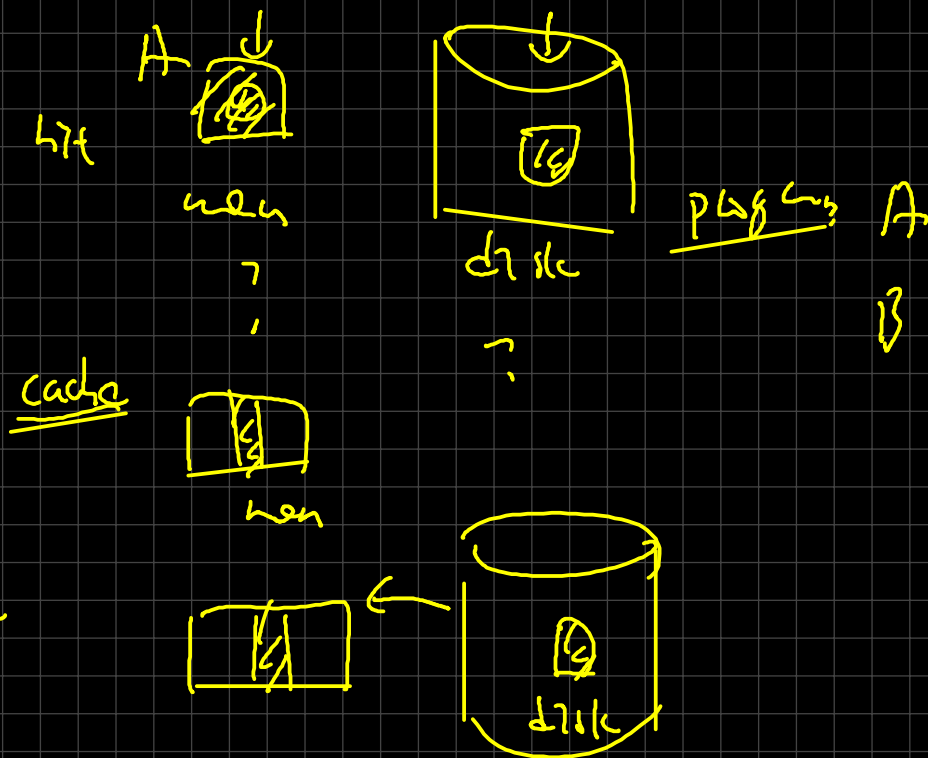
key

Page table



$(VPN \rightarrow PFN) + \text{page offset}$   
 $\text{PA}$

(1) page 有可能在 memory, 也有可能 disk



(miss)  
 fault page

Overall

VM



CPU 使用 Virtual Address

VA 不能直接代表 RAM 位置

PA

需要 Address Translation

Page Table: VPN → PPN

→ PA

access mem

但是每次查 Page Table 太慢

TLB: cache 最近的 translation

得到 Physical Address

CPU  
access

用 PA access Cache

但是「先 TLB → 再 Cache」太慢

想讓 TLB + Cache 平行

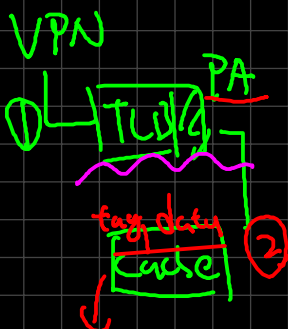
VIPT

又會牽涉 page offset / cache index 限制

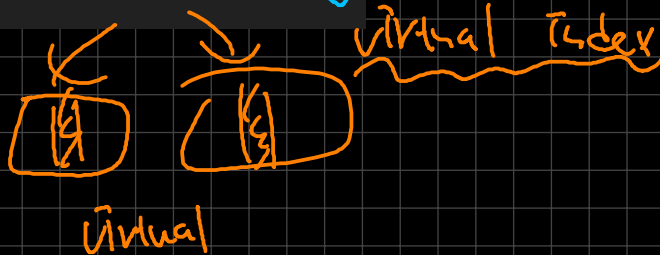
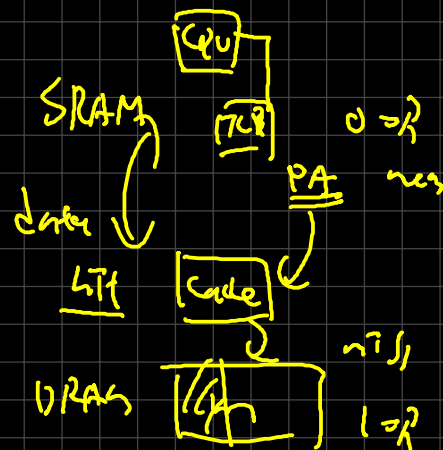
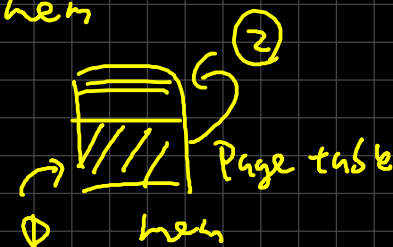
alias synonym 問題

alias

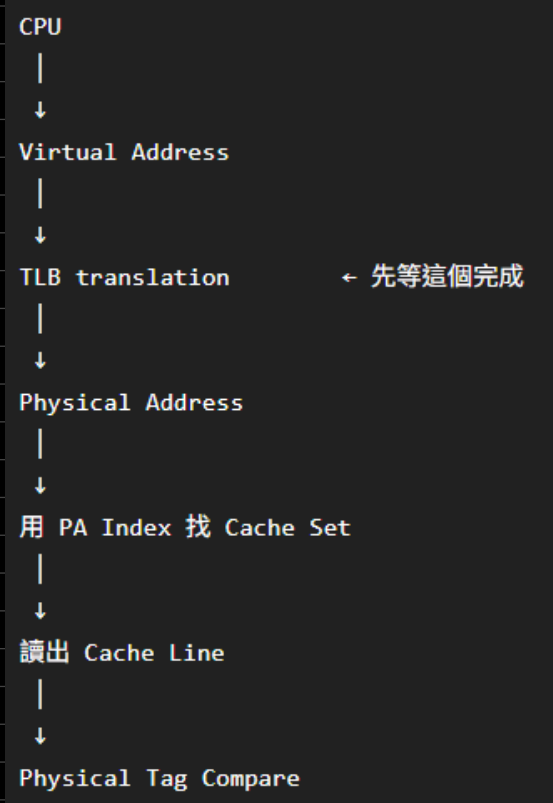
TLB  
assoc  
(associative)  
memory



seg  
↓  
puba



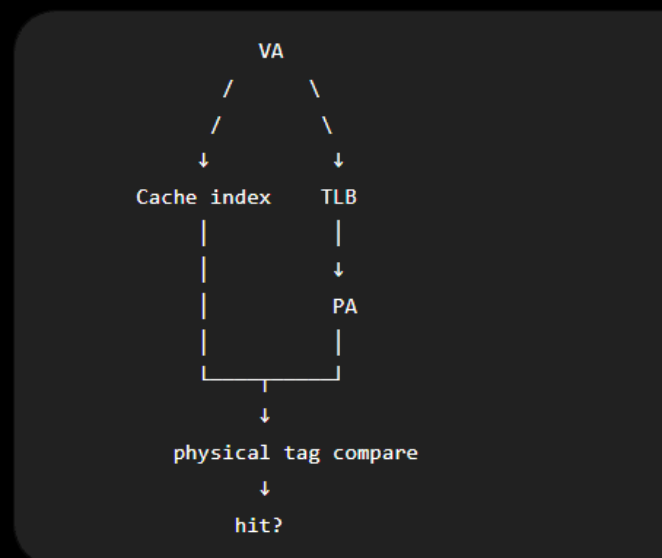
## Physical Indexed, Physical Tagged



## Virtual Indexed, Physical Tagged

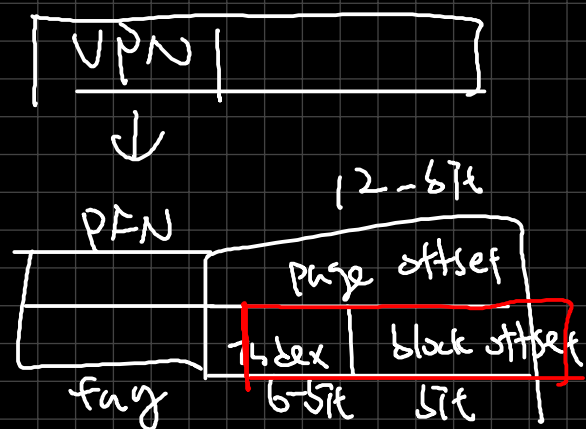
VIPT = Virtually Indexed, Physically Tagged

概念變成：



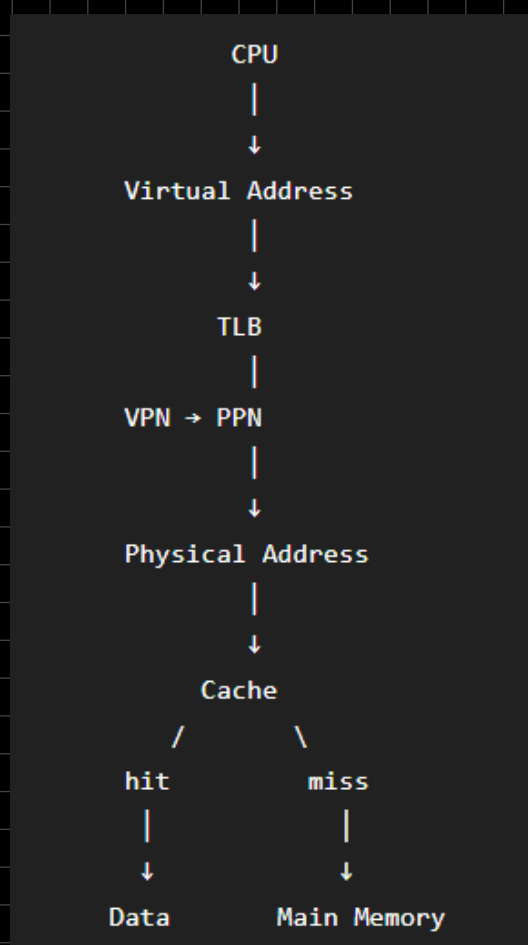
✓ VIPT 還是做 translation，只是 Cache indexing 不需要等 translation。

## Virtual address cache



page offset  $\geq$   
Index + block offset

✓



virtual index

Virt

VA  
PA

台大 97

p.161

4. [10 pts] For the following memory system parameters:

- Cache: virtually-addressed but physically-tagged, 8KB, direct-mapped, 32B block size
- TLB: fully associative, 32 entries
- 32-bit physical address
- Virtual Memory: 8K page, 64-bit virtual address

(a) (5 pts) How many bits are needed in each tag, index, and block offset fields of the cache?

(b) (2 pts) How many bits are needed in each tag field of the TLB?

(c) (3 pts) Why do we want to use a virtually-addressed but physically-tagged cache?

台大 111

14. (5 pts) Assume a virtual memory system with an 4KiB page size. The cache capacity is 16KiB, and the block size is 8-word (1 word = 4 bytes). Which cache architecture could be implemented as a virtually-indexed, physically tagged cache:

- (A) Direct-mapped cache
- (B) 2-way associative cache
- ✓ (C) 4-way associative cache
- ✓ (D) 8-way associative cache
- ✓ (E) 16-way associative cache

$\text{Index bits} + \text{Block Offset bits} \leq \text{Page Offset bits}$